



FileFlow Studio

User Manual and Reference Guide

FileFlow Studio — Motor Visual de Automatización DAG en .NET 9



User Manual & Node Reference Guide

FileFlow Studio v2.0

Modular File Automation, Batch Processing & DAG Pipeline Platform

Runtime .NET 9 | C# 13 | GNU GPLv3 License | Copyright © 2026 RGLara



Table of Contents

1. Introduction & Architectural Philosophy
2. Visual Editor Core Concepts
 - Interactive Node Canvas (Nodify)
 - File Item Pipeline Context (`FileItemContext`)
 - Nested Sub-workflows & Macros (Breadcrumbs)
 - Real-time Connection Telemetry Badges
 - QuickLook Previewer & Node Inspector
3. Execution Modes & Data Safety
 - Standard Parallel Execution
 - Virtual Simulation Mode ("Dry Run")
 - Continuous Real-time Watchdog Mode
 - Transactional LIFO Rollback System
 - Interactive Debugging with Breakpoints
 - Virtual File System (VFS) & Non-Destructive Test Bench
 - Synthetic Data Set Designer
 - Hybrid Compressed Archive Simulation
4. Token & Dynamic Template Engine

- Syntax & Token Domains
- Comprehensive Token Reference

5. Complete Node Catalog (58 DAG Nodes)

- 📁 Category 1: FileSystem (Disk I/O & Lifecycle)
- 🗄️ Category 2: Archives (Compression & Extraction)
- 🖼️ Category 3: Images (Graphic Processing & EXIF)
- 🌐 Category 4: Network & Remote Storage (Multi-Protocol Hubs)
- 🤖 Category 5: AI & Machine Learning (Local ONNX Models)
- 📄 Category 6: Documents & PDFs (Splitting, Merging & Extraction)
- 📊 Category 7: Data & Tabular Files (Excel, CSV, SQLite)
- ⚙️ Category 8: Logic & Control Flow (Routing & Synchronization)
- 🔒 Category 9: Hashing & Security (Cryptography & Deduplication)
- 📜 Category 10: Scripting & Extensibility (C# Roslyn & JS)
- 🔧 Category 11: Integrations & CLI (External Tools & FFmpeg)

6. Step-by-Step Educational Tutorials

- Tutorial A: Automated Photo Organization & Optimization
- Tutorial B: Remote SFTP Ingestion, Unpacking & Consolidated Report
- Tutorial C: AI Pipeline with Local OCR & PII Anonymization

7. Keyboard Shortcuts & Productivity

1. Introduction & Architectural Philosophy

FileFlow Studio is a modern visual workflow automation engine and batch file transformation platform inspired by state-of-the-art tools like *n8n*, *ComfyUI*, and *Node-RED*, engineered specifically on **.NET 9** and **C# 13**.

Core Tenets:

- 🛡️ **Non-Destructive by Default:** Input files (`OriginalPath`) are never modified or destroyed in place. Source file lifecycle manipulation (preserving, quarantining, recycling, or deleting) is strictly delegated to the specialized `OriginalFileActionNode`.
- 🧩 **Decoupled Microkernel Architecture (ADR-006):** Every plugin domain (`FileFlow.Plugin.*`) is completely autonomous with zero UI dependencies and co-located localization resources (`Strings.resx` / `Strings.es.resx`).
- ⚡ **High-Performance Asynchronous Channels:** Multi-threaded parallel processing powered by `System.Threading.Channels` and `TPL Dataflow`, achieving over **82,000 telemetry events/second**.
- 🌐 **Universal Network Symmetry:** Unified download and upload hubs covering **HTTP/HTTPS**, **FTP/FTPS**, **SFTP/SSH**, **WebDAV/Nextcloud**, and **SMB/Windows Shares** with reactive parameter

visibility.

2. Visual Editor Core Concepts

Interactive Node Canvas (Nodify)

Model pipelines by dragging and connecting nodes from the **Toolbox**:

- **Input Ports (Left Edge)**: Receive incoming items (`In` , `BranchA` , `Files`).
- **Output Ports (Right Edge)**: Emit transformed files or conditional branch items (`Out` , `Done` , `Error` , `Matched` , `Unmatched`).
- **Status LED**:
 -  *Idle*: Waiting for input items.
 -  *Pulsing Blue (Running)*: Processing files in real time.
 -  *Green (Completed)*: Successfully finished batch.
 -  *Red (Faulted)*: Error captured (routed to `Error` port without halting the pipeline).
- **Breakpoint Toggle**: Click the red circle icon to pause execution when a file arrives at the node.

File Item Pipeline Context (`FileItemContext`)

The fundamental data record travelling across workflow connections:

```
public sealed record FileItemContext
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public string CurrentPath { get; set; } // Current file path in the pipeline step
    public string OriginalPath { get; init; } // Immutable source path
    public long FileSizeBytes { get; set; } // Exact byte size
    public bool IsDirectory { get; set; }
    public Dictionary<string, object?> Metadata { get; } // Enriched key-value data (EXIF, Hash, OCR, ...)
    public HashSet<string> Tags { get; } // Quick taxonomy tags
    public List<string> ExecutionLog { get; } // Audit transformation trail
}
```

Nested Sub-workflows & Macros (Breadcrumbs)

Encapsulate complex pipelines into reusable composite nodes:

1. Double-click a sub-workflow node to enter its internal editor.
2. The top breadcrumb bar tracks hierarchy: `Root Workflow > Extraction Macro > Sanitization` .
3. Click any parent breadcrumb to commit changes and return to the main canvas.

Real-time Connection Telemetry Badges

Each wire dynamically renders a live counter badge (e.g.,  2,450 items) showing item flow rates and eliminating bottlenecks instantly.

QuickLook Previewer & Node Inspector

- Press  or click  QuickLook on any node or log entry to inspect images, documents, raw text, or metadata tables.
- The **Node Inspector Panel** exposes parameters with rich controls (file pickers, sliders, checkboxes, and reactive protocol dropdowns).

Opening and Saving Flows: the File Format Version

Every saved flow declares the format it was written in, and the application takes care of the rest:

- **A flow saved with an older version of the format is repaired when opened.** Design data that version never stored—for example, the ports a sub-workflow container exposed—is recovered from its own wires, and anything that could not be rebuilt is reported in the console and in the status bar instead of being dropped silently.
- **Saving it moves the file to the current format**, so it stops being repaired: open once, save, and the file is up to date. Right after opening and before saving, the file on disk is left as it was.
- **A flow written by a newer version is never overwritten.** It opens in full, and if you try to save over it the application warns you and offers another path, so the fields that version added are not lost.

3. Execution Modes & Data Safety

Standard Parallel Execution

Press "► **Run Workflow**" () to process files using hardware-aware adaptive concurrency.

Virtual Simulation Mode ("Dry Run")

1. Check "**Dry Run**" before execution.
2. The engine computes paths, checks conditions, generates hashes, and resolves templates **without writing, moving, or deleting any files on disk.**
3. Inspect planned operations () in the console prior to committing.

Continuous Real-time Watchdog Mode

- Toggle "  **Watchdog Mode** " to monitor input folders for new files and trigger processing immediately on disk events.

Transactional LIFO Rollback System

If you need to revert changes:

1. Click " **Rollback**" on the control bar.
2. Operations (renames, relocations) are undone in reverse order (Last-In, First-Out).

Interactive Debugging with Breakpoints

- Click " **Debug Workflow**".
- Execution halts when an item reaches a breakpoint.
- Use "**Step Into (F10)**" to advance node by node while observing live metadata diffs in the inspector.

Virtual File System (VFS) & Non-Destructive Test Bench

To design, validate, and debug complex pipelines without manipulating real disks or endangering production files:

1. Automated Zero-Config Activation:

- Adding a `SyntheticDataSourceNode` or tagging items as virtual (`IsVirtual = true`) causes the DAG engine (`WorkflowExecutor`) to transparently instantiate an in-memory Virtual File System store (`IVirtualFileSystemStore`).
- File sink and relocation nodes (`DestinationSinkNode` , `FileRelocatorNode` , `SafeRecycleDeleteNode` , `OriginalFileActionNode`) automatically detect the virtual execution context and redirect their writes, copies, moves, and logical deletions to the VFS store without I/O errors or cluttering local storage.

2. Visual VFS Explorer (`VirtualFileSystemExplorerWindow`):

- Once a run completes with virtual items, the control bar displays a reactive badge button  **VFS (N)** indicating total virtual files generated. It is also permanently accessible via the side navigation Drawer.
- **3-Pane Split View:**
- *Directory Tree*: Interactive recursive folder hierarchy created in memory.
- *File Table*: Detailed view with operation badges (`Saved` , `Copied` , `Moved` , `ConflictRenamed` , `Recycled` , `Deleted`).
- *Metadata Inspector*: Right sidebar categorized by domain (Photo/EXIF, Music/Audio ID3, Video, Documents/Fiscal, and Cryptographic SHA/MD5 hashes).
- **Export & Diagnostics Tools:**
-  `Copy ASCII Tree` : Copies a formatted ASCII tree diagram to the clipboard.
-  `Open in Explorer` : Safely exports the in-memory VFS hierarchy into a temporary sandbox folder (`%TEMP%/FileFlow_VFS_Sandbox/ ...`) and opens it directly in Windows File Explorer.

Synthetic Data Set Designer (`SyntheticDataSetDesignerWindow`)

Allows users to create, modify, persist, and reuse custom test datasets with customized directory trees:

• Direct Entry Points:

1.  `Diseñar Conjuntos de Datos ...` custom action button in `SyntheticDataSourceNode` inspector.
2.  `Diseñador ...` button on the sample toolbar in `AdvancedRenamerEditorWindow` .

3. **Diseñador de Datos Sintéticos** menu item in the main window Drawer.

- **Dataset Catalog & Sidebar:**

- Live search filter by name and category.
- Counters showing item count, directory count, and simulated archives.
- Create (**+ New**), Duplicate (**Duplicate**), and Delete (**Delete**) actions with protection for built-in system datasets (**IsBuiltIn**).
- Thread-safe persistence in `%AppData%/FileFlow/SyntheticDataSets/*.json`.

- **Three Synchronized Editing Modes:**

- **Visual Table:** Cell-by-cell editing of relative paths (**RelativePath**), byte sizes, **Is Directory** and **Is Archive** flags, and metadata dictionaries.
- **Fast Tree (DSL):** Lightning-fast indented text editor with a bidirectional hierarchical parser. Press **Apply Changes** to sync with the visual table.
- **Raw JSON:** Inspect, format, copy, or paste raw JSON dataset definitions.

Fast Tree DSL Syntax:

```
# Hierarchical tree definition with metadata and inner archive entries
Photos/
  2026/
    beach.jpg (3.5MB, Exif:CameraModel=Sony A7 IV, Exif:ISO=100)
    trip.zip (15MB) [archive: track.gpx (12KB); notes.txt (5KB)]
Documents/
  Invoices/
    Invoice_001.pdf (250KB, Doc:Author=Accounting, FiscalYear=2026)
Music/
  Daft Punk/
    Discovery/
      01 - One More Time.flac (35MB, Audio:Artist=Daft Punk, Audio:Track=01)
```

- **Directories:** End with `/` or omit file extensions.
- **Human-readable Sizes:** Declared inside parentheses `(500B)`, `(15KB)`, `(3.5MB)`, `(1.2GB)`.
- **Inline Metadata:** Key-value pairs inside parentheses: `(2MB, Audio:Artist=Queen)`.
- **Inner Archive Entries:** Declared at the end of the line using `[archive: inner1.txt (500B); inner2.png (2MB)]`.

Hybrid Compressed Archive Simulation (ZIP, RAR, 7Z)

- **Direct Virtual Extraction (VFS):** When a simulated archive item flows into **SmartUnpackNode**, the node recognizes that the item is virtual or contains **Archive:Entries** metadata and unpacks its internal entries directly into **IVirtualFileSystemStore** preserving subfolder hierarchies and individual metadata tags, without physical disk reading.
- **Real File Generation in PhysicalMock Mode:** When **SyntheticDataSourceNode** runs in physical mode, it automatically generates a real lightweight `.zip` archive via `System.IO.Compression.ZipArchive`

containing the simulated inner files, allowing testing of external CLI unpackers and third-party tools against physical mock data.

4. Token & Dynamic Template Engine

The `VariableTemplateResolver` engine allows powerful token substitution in filenames, folders, payloads, and CLI parameters.

Syntax & Token Domains

`{Domain:Key:Modifier}` or `{Variable}`

Comprehensive Token Reference

Token	Output Example	Description
<code>:---</code>	<code>:---</code>	<code>:---</code>
<code>{FileName}</code>	<code>report.pdf</code>	Full filename with extension
<code>{FileNameNoExt}</code>	<code>report</code>	Filename without extension
<code>{Ext}</code>	<code>pdf</code>	Lowercase extension without dot
<code>{ParentDir}</code>	<code>Invoices_2026</code>	Name of parent directory
<code>{CreationDate:yyyyMMdd}</code>	<code>20260903</code>	Formatted file creation date
<code>{ModifiedDate:yyyy-MM-dd}</code>	<code>2026-09-03</code>	Formatted last modification date
<code>{Now:yyyyMMdd_HH:mm:ss}</code>	<code>20260903_205000</code>	Current system timestamp
<code>{FileSize:MB}</code>	<code>14.50</code>	Formatted file size in Megabytes
<code>{FileSize:KB}</code>	<code>14848.0</code>	Formatted file size in Kilobytes
<code>{Hash:SHA256}</code>	<code>e3b0c44298 ...</code>	Full SHA-256 cryptographic checksum
<code>{Hash:SHA256:8}</code>	<code>e3b0c442</code>	SHA-256 hash truncated to 8 characters
<code>{Hash:MD5:6}</code>	<code>d41d8c</code>	MD5 hash truncated to 6 characters
<code>{Exif:CameraModel}</code>	<code>Nikon Z8</code>	Camera model from EXIF metadata
<code>{Exif:DateTimeOriginal}</code>	<code>2026:08:15 14:20:00</code>	Original image capture timestamp
<code>{Ocr:Text}</code>	<code>Invoice #1024</code>	Text extracted via local OCR engine
<code>{Env:USERPROFILE}</code>	<code>C:\Users\User</code>	Operating system environment variable
<code>{Meta:CustomKey}</code>	<code>DynamicValue</code>	Metadata injected by upstream nodes

5. Node Catalog by Category

This section describes the most used nodes. The **complete, up-to-date** catalog —ports, parameters and a link to each node's source— lives in `.agents/nodes_catalog.md`, which is generated from the code itself and kept honest by a guard.

Category 1: FileSystem

1. **FolderSourceNode** : Discovers and emits files with extension filters, recursive scanning, and real-time folder watching.
2. **DestinationSinkNode** : Consolidates processed files with collision strategies (`Overwrite` , `Skip` , `RenameIncremental`) and transparent non-destructive VFS support.
3. **AdvancedRenamerNode** : Batch file renaming with token templates, character sanitization, and live preview.
4. **FileRelocatorNode** : Moves, copies, or hard-links files to calculated destinations with optional SHA-256 validation and VFS redirection during testing.
5. **SafeRecycleDeleteNode** : Safe recycling by sending files to the Windows Recycle Bin (`SHFileOperationW`) or performing logical deletion in VFS.
6. **OriginalFileActionNode** : Centralized lifecycle management for source files (`Keep` , `MoveToRecycleBin` , `MoveToQuarantine`).
7. **OperationReportNode** : Generates interactive reports in `HTML` , `Markdown` , `Text` , `JSON` , or `CSV` format with full audit history.
8. **DirectoryInspectorNode** : Classifies folder structure (single archive vs. mixed contents).
9. **EmptyDirectoryCleanerNode** : Cleans up leftover empty directory trees after moving or unpacking files.
10. **DocumentProcessorNode** : Unified metadata extractor for `.pdf` , `.docx` , `.txt` , `.csv` , and `.json` documents.
11. **VariableInjectorNode** : Injects custom variables and calculated expressions into file metadata.
12. **LogOutputNode** : Emits custom formatted diagnostic log messages to the console.
13. **FileAttributeNode** : Modifies file system attributes (ReadOnly, Hidden, Archive, Timestamps).
14. **PathSplitterNode** : Decomposes paths into discrete token variables.
15. **SyntheticDataSourceNode** : Hyper-realistic synthetic test bench and dataset generator. Simulates collections (Movies, TV Series, Music with ID3, Photos with EXIF, Documents/Invoices, or custom user datasets) in memory (`Virtual` mode over VFS) or in a temporary folder (`PhysicalMock`). Supports hierarchical folder emission (`EmitDirectories`), configurable emission delays (`EmissionDelayMs`), and hybrid compressed

archive simulation with instant virtual unpacking in `SmartUnpackNode`. Features a custom action button to open the **Synthetic Data Set Designer**.

Category 2: Archives

1. `SmartUnpackNode`: Universal archive extraction (ZIP, RAR, 7Z, TAR, GZ) with redundant folder flattening and Zip Slip protection.
 2. `ArchiveCompressorNode`: Compresses single or batch files into ZIP, 7Z, TAR, or GZ archives with configurable compression algorithms. **Where it writes:** to the output folder of the flow—the one the flow declares as its own, and the default output folder from the settings when the flow declares none—and the `Destination Folder` parameter is for saying something else: a subfolder of your own, a full path, or `{CurrentDir}` to leave the archive next to the file it compresses. The run log says where it ended up.
 3. `ArchiveFilterNode`: Detects and isolates split multi-volume archives (`.part1.rar` , `.z01`).
-

Category 3: Images

1. `ImageOptimizerNode`: Resizes, optimizes, and converts images to WebP, JPEG, or PNG, reporting exact byte savings.
 2. `ExifMetadataNode`: Extracts EXIF camera metadata (make, model, GPS coordinates, date taken).
 3. `ImageWatermarkNode`: Applies text or image watermarks with configurable positioning, opacity, and scaling.
 4. `ImageMetadataStripperNode`: Strips sensitive GPS and camera metadata prior to publishing.
-

Category 4: Network & Remote Storage

1. `NetworkDownloadNode` (*Universal Download Hub*):
 - Supports 5 symmetric protocols: **HTTP/HTTPS**, **FTP/FTPS**, **SFTP (SSH)**, **WebDAV (Nextcloud/ownCloud)**, and **SMB (Windows Network Shares / NAS)**.
 - Dynamic parameter visibility adapting in real time to the selected protocol.
 2. `NetworkUploadNode` (*Universal Upload & Transfer Hub*):
 - Supports 5 symmetric protocols: **HTTP POST/PUT**, **FTP/FTPS**, **SFTP (SSH)**, **WebDAV**, and **SMB**.
 - Resilient transfers with auto-retry, password/private key SSH authentication, and remote directory creation.
-

Category 5: AI & Machine Learning

1. `SmartImageClassifierNode`: Offline image classification using local ONNX neural networks (e.g. ResNet, MobileNet).

2. **PromptObjectDetectorNode** : Text-prompted zero-shot object detection via YOLO-World or Grounding DINO in ONNX.
 3. **LocalOcrNode** : Fast offline optical character recognition for scanned receipts, documents, and images.
 4. **WhisperAudioTranscriberNode** : High-accuracy local speech-to-text audio/video transcription with OpenAI Whisper ONNX.
 5. **FaceDetectorNode** : Detects faces in images, calculating bounding boxes and face counts.
 6. **ZeroShotSemanticSearchNode** : Semantic text classification and document routing without fine-tuning.
 7. **PiiAnonymizerNode** : Detects and redacts personally identifiable information (SSN, credit cards, names, emails) using NER models.
 8. **SuperResolutionUpscalerNode** : Deep learning image upscaling and super-resolution enhancement.
-

Category 6: Documents & PDFs

1. **PdfMergeNode** : Merges multiple PDF files into a single master document.
 2. **PdfSplitNode** : Splits multi-page PDF documents into single pages or specified page ranges.
 3. **PdfTextExtractorNode** : Extracts structured text and layout data from PDF files using **PdfFig** .
 4. **PdfMetadataNode** : Reads and updates standard PDF metadata (Title, Author, Subject, Keywords).
-

Category 7: Data & Tabular Files

1. **ExcelReaderNode** : High-performance streaming reader for **.xlsx** / **.xls** spreadsheets via **MiniExcel** .
 2. **CsvProcessorNode** : Parses, filters, converts, and formats delimited text files (CSV/TSV).
 3. **DataLookupNode** : Instant $O(1)$ in-memory relational lookup against master reference tables.
-

Category 8: Logic & Control Flow

1. **SwitchCaseNode** : Multi-way conditional branch router matching extensions, file sizes, or metadata.
2. **ExpressionFilterNode** : Boolean predicate filter with logical operators (**Equal** , **Contains** , **GreaterThan** , **RegexMatch**).
3. **BatchBufferNode** : Gathers items until the configured batch size or total size is reached, then releases them together; a batch that never fills is still delivered when the execution ends.
4. **ThrottleDelayNode** : Rate limits emission rate to prevent overwhelming downstream disk or network endpoints.
5. **ForkJoinBarrierNode** : Synchronizes concurrent branches, waiting for all sibling items to arrive before releasing.

6. **VariableInjectorNode** : Injects static and dynamic variables into item context.
-

Category 9: Hashing & Security

1. **HashCalculatorNode** : Computes cryptographic checksums (SHA-256, SHA-512, MD5, SHA-1, xxHash).
 2. **DeduplicationFilterNode** : Filters duplicate files in real time by comparing memory-cached hash signatures.
 3. **ChecksumVerifierNode** : Validates files against `.sha256` checksum files or expected metadata hashes.
-

Category 10: Scripting & Extensibility

1. **CustomScriptNode** : Custom logic execution with dual support for **C# (Roslyn JIT)** and **JavaScript (Jint sandbox)**.
 2. **ScriptStudio** : Integrated IDE with syntax highlighting, live testing console, and `.ffscript` templates.
 3. **PythonScriptNode** : Dispatches items through external Python environments for specialized data science workloads.
-

Category 11: Integrations & CLI

1. **CliExecutionNode** : Executes command-line processes (PowerShell, CMD, binaries) capturing stdout/stderr into metadata.
 2. **WebhookNotificationNode** : Dispatches HTTP POST/PUT alerts with custom JSON payloads to Discord, Slack, or webhook endpoints.
 3. **MediaTranscoderNode** : Transcodes video and audio streams using integrated FFmpeg profiles.
 4. **SQLiteDatabaseSinkNode** : Inserts structured audit and file records into local SQLite database tables.
 5. **MessageQueuePublisherNode** : Publishes event metadata to message queues (RabbitMQ, MQTT).
-

6. Step-by-Step Educational Tutorials

Tutorial A: Automated Photo Organization & Optimization

Goal: Scan an SD card, extract EXIF data, convert images to modern WebP, and organize them into folders by year, month, and camera model.

1. **FolderSourceNode** :
 - **SourcePath** : `E:\DCIM\100NIKON`
 - **ExtensionFilter** : `.jpg, .jpeg, *.png`
2. Connect **Out** to **ExifMetadataNode** (extracts camera make, model, and date).

3. Connect **Out** to **ImageOptimizerNode** :

- **TargetFormat** : WebP
- **Quality** : 85

4. Connect **Out** to **DestinationSinkNode** :

- **DestinationRoot** : D:\Organized_Photos\{Exif:Make}_{Exif:CameraModel}\{CreationDate:yyyy}\{CreationDate:MM}
- **ConflictStrategy** : AutoIncrement

5. Connect **Done** to **OriginalFileActionNode** :

- **ActionType** : MoveToQuarantine (safely backs up source files).
-

Tutorial B: Remote SFTP Ingestion, Unpacking & Consolidated Report

Goal: Download daily backups from an SSH server, extract contents, discard duplicate files, and produce an interactive HTML audit report.

1. **NetworkDownloadNode** :

- **Protocol** : SFTP
- **Host** : backup.company.com | **Username** : backup_operator
- **RemoteFilePath** : /var/backups/nightly.zip
- **DestinationFolder** : C:\Temp\Ingest

2. Connect **Out** to **SmartUnpackNode** (extracts all nested files).

3. Connect **Out** to **HashCalculatorNode** (**Algorithm**: SHA256).

4. Connect **Out** to **DeduplicationFilterNode** :

- **Unique** output \$\rightarrow\$ Connect to **DestinationSinkNode** (**DestinationRoot**: D:\Clean_Store).
- **Duplicate** output \$\rightarrow\$ Connect to **SafeRecycleDeleteNode** (sends to Windows Recycle Bin).

5. Connect **Unique** to **OperationReportNode** :

- **ReportFormat** : HTML
 - **AutoOpenReport** : true
-

Tutorial C: AI Pipeline with Local OCR & PII Anonymization

Goal: Ingest confidential scanned invoices, perform local OCR, and anonymize sensitive personal data before archiving.

1. **FolderSourceNode** (**SourcePath**: C:\Incoming_Invoices).

2. Connect to **LocalOcrNode** (extracts text without sending data to cloud APIs).

3. Connect to **PiiAnonymizerNode** (masks credit card numbers, IDs, and names).
4. Connect to **DestinationSinkNode** (**DestinationRoot**: C:\Archived_Invoices).

7. Keyboard Shortcuts & Productivity

Shortcut	Action
:---	:---
F5	Run current workflow
Ctrl + F5	Run in Virtual Simulation Mode (Dry Run)
F10	Step Into next node during debugging
Ctrl + Z	Undo last canvas action
Ctrl + Y	Redo last canvas action
Ctrl + S	Save current workflow (.json)
Ctrl + O	Open workflow file
Ctrl + N	Create new blank workflow
Spacebar	Open QuickLook preview for selected item
Delete	Delete selected node or connection
Ctrl + F	Search nodes in Toolbox
Ctrl + Mouse Wheel	Zoom in / Zoom out on canvas

Official FileFlow Studio Documentation. Distributed under GNU General Public License v3.0.